SEMPRE: Semantic-Aware Protocol Reverse Engineering via Consistency-Driven Joint Optimization

Yihua Xu
Illinois Institute of Technology
yxu131@hawk.illinoistech.edu

Weijie Huang
Indiana University
wh25@iu.edu

Xiaofeng Wang
Nanyang Technological University
xiaofeng.wang@ntu.edu.sg

Hang Zhang
Indiana University
hz64@iu.edu

Binghui Wang
Illinois Institute of Technology
bwang70@illinoistech.edu

Yue Duan
Singapore Management University
yueduan@smu.edu.sg

arXiv:2610.31844v1 [cs.CR] 22 Oct 2026

***Abstract*—Protocol reverse engineering (PRE) is essential for understanding undocumented or proprietary communication formats, enabling security-critical tasks such as vulnerability discovery, malware C2 analysis, and IDS signature generation. However, existing PRE techniques rely on brittle hand-written rules, under-model contextual dependencies among neighboring fields, and often fail on messages with nested sub-protocols. We present SEMPRE, a fully automated PRE framework that replaces ad-hoc heuristics with recursive LLM reasoning grounded in fine-grained dynamic program analysis. SEMPRE uses taint-flow and loop traces to guide an LLM-driven tree-of-thought search that slices messages into fields while detecting nested sub-payloads, producing a hierarchical layout. It then infers each field's type and role and iteratively refines boundaries by freezing high-confidence segments and re-running detection under semantic constraints, yielding an EM-style procedure that monotonically improves structure and semantics until convergence. Evaluated on a broad corpus of mainstream protocols against six state-of-the-art PRE systems, SEMPRE achieves an average F1 of 0.76 for field detection (vs. <0.40 for all baselines) and 0.72 for semantic inference (vs. 0.14 for BinPRE). Ablations confirm the contribution of each component, and a case study on a prolific real-world botnet demonstrates practical utility.**

I. INTRODUCTION

Protocol reverse engineering (PRE) is the process of analyzing communication flows to deduce the message structures, semantics, or state machines of network protocols, typically when official specifications are unavailable, incomplete, or proprietary [1], [2]. When only protocol messages and the program binaries that implement the protocol are available, PRE becomes particularly challenging. Analysts often have to rely on captured network traffic and the behavior of related (often closed-source) software to infer how systems exchange information. Figure 1 illustrates the expected output of PRE on a real-world Modbus message [3]. Given a 12-byte message (byte [0] to byte [11]), PRE aims to: (1) accurately identify the boundaries of all fields within the message (field detection), and (2) infer each field's type and role (semantic inference). PRE is a key enabler for security applications, including analyzing malware command-and-control (C2) communications [4], [5], discovering vulnerabilities in undocumented or proprietary protocols [6], and generating intrusion detection system (IDS) signatures for previously unknown threats [7].

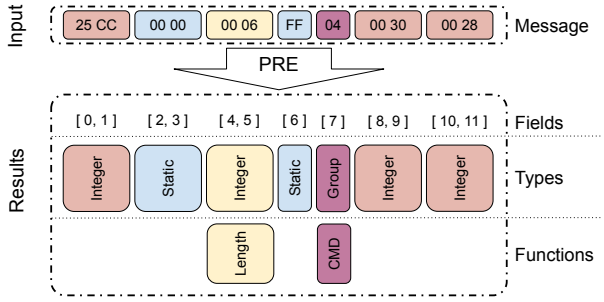


Fig. 1: Protocol Reverse Engineering

Due to the importance of PRE, a substantial body of research has emerged. Existing studies generally fall into two categories: network-traffic-based approaches and binary-analysis-based approaches. Network-traffic-based methods, such as BinaryInferno [8], NEMESYS [9], DynPRE [10], and NetPlier [11], take captured network traces as input and apply statistical techniques to extract recurring format patterns. However, their effectiveness depends heavily on access to high-quality and diverse traffic that captures many protocol message instances—an assumption that is often difficult to satisfy in practice. In contrast, binary-analysis-based approaches, including Dispatcher [12], PolyGlot [13], Tupni [14], and BinPRE [2], leverage dynamic program analysis to recover detailed runtime semantics by observing how the protocol implementation parses and consumes message bytes. For example, if the binary consistently loads bytes [0–1] together, applies endian-aware integer operations (e.g., shifting/combining), and uses the resulting value in conditional branches or length calculations, this provides strong evidence that [0–1] forms a multi-byte integer field rather than unrelated bytes. As a result, binary-based approaches are often more practical in real-world settings where high-quality traffic traces are unavailable. Despite significant progress, we identify three fundamental

Network and Distributed System Security (NDSS) Symposium 2027
22–26 March 2027, Seoul, Republic of Korea
ISBN 978-1-970672-09-1
https://dx.doi.org/10.14722/ndss.2027.242825
www.ndss-symposium.org

limitations of state-of-the-art (SOTA) binary-analysis-based techniques, as elaborated below.

First, SOTA approaches rely heavily on *hand-crafted heuristics* or *rule sets* that are tuned to specific byte patterns or protocol families. This rule-driven design limits generalizability and makes it difficult to keep pace with the diversity of real-world encodings. A representative example is BinPRE [2], which uses multiple rules to label a field as a filename only when the byte sequence ends with a known extension (e.g., ".txt" or ".jpg"). Consequently, filenames that are length-prefixed, UTF-16 encoded, or extensionless can be missed or mislabeled.

Second, existing techniques often *model fields in isolation*, without explicitly capturing semantic relationships among neighboring fields. In practice, many protocols exhibit strong contextual dependencies that are crucial for semantic inference. For instance, a tag immediately followed by a length field strongly suggests the canonical Type–Length–Value (TLV) pattern, and checksum fields typically appear after the data they protect. The Dynamic Host Configuration Protocol (DHCP) [15], for example, encodes options as a sequence of TLV triples. Reliably identifying the Tag–Length prefix is essential for delineating the corresponding value fields and inferring their types. Prior work, such as BinaryInferno [8], can leverage serialization patterns to detect certain LV/TLV-like structures, but it does not reason about *why* fields depend on one another or how upstream boundary errors propagate to downstream semantic inference. As a result, failing to model such dependencies can lead to inaccuracies that may cascade into subsequent analyses, such as vulnerability discovery.

Third, these PRE techniques struggle to robustly handle a special yet prevalent case – *nested protocols*, where a message embeds a sub-payload that follows a different protocol specification. While dynamic binary-analysis–based approaches (e.g., Polyglot [13], Tupni [14], and BinPRE [2]) can expose certain field semantics by observing runtime parsing behavior or can infer repeated or nested layouts from control-flow regularities, these methods primarily recover syntactic structure rather than an explicit *hierarchical* semantic representation. In practice, inferred fields are often flattened into a single sequence, obscuring protocol-layer boundaries. However, nested payloads typically induce stronger dependencies *within* a sub-protocol than *across* protocol boundaries. Existing systems do not explicitly represent or reason about this distinction, and approaches with limited semantic cues (e.g., BinaryInferno) still assign roles largely at the level of individual fields, without modeling hierarchical relationships or distinguishing intra-payload dependencies from inter-layer transitions. Consequently, nested protocols may be only partially recovered, and inferred layouts can become brittle for multi-layer designs such as EtherNet/IP (EIP) [16] encapsulating Common Industrial Protocol (CIP) [17] payloads.

Recent studies [18], [19], [20] suggest that large language models (LLMs) can outperform rigid, heuristic-based pipelines in reverse-engineering tasks such as disassembly [20] and decompilation [19], [18]. Trained on large corpora, LLMs can internalize recurring structural regularities and, importantly, capture relationships among distant components, enabling more flexible inference under uncertainty. Motivated by these advances, we first examined whether an off-the-shelf LLM could perform protocol reverse engineering (PRE) without additional structure. In a pilot study, we asked GPT-o3 (the same backbone model used in our approach) to perform field detection on 179 messages from four representative protocols (50 Modbus, 41 S7, 38 FTP, and 50 DNS). The results exposed the limitations of naïve prompting: the model achieved an F1 of 0.69 on Modbus (the simplest format in our set) and remained below 0.50 on S7, FTP, and DNS. These outcomes indicate that dependable PRE requires more than a single, unconstrained pass through an LLM. A qualitative inspection suggests that naïve prompts often trigger hallucinated fields, incorrect semantic assumptions, and inconsistent labeling across messages, especially for formats with optional fields, variable-length encodings, or layered structure. We attribute this behavior to the difficulty of reliably grounding generation on long, noisy inputs (e.g., dynamic traces) and maintaining global consistency when many interacting constraints must be satisfied simultaneously.

To effectively tackle the PRE problem, our key insight is that *field detection and semantic inference are tightly coupled*: accurate boundaries make semantics easier to infer, while semantic constraints (e.g., TLV relationships, checksums, length fields, nested payload headers) can in turn validate and refine boundaries. With this, we develop SEMPRE, a fully automated PRE framework, which integrates fine-grained dynamic program analysis with a recursive LLM-driven reasoning procedure that performs *joint* optimizations over field detection and semantic inference, rather than treating them as two independent stages as in prior work. SEMPRE first performs fine-grained dynamic taint analysis and loop detection while executing the protocol binary on a given input message. These traces provide low-level evidence about how individual bytes are consumed (e.g., grouped loads, comparisons, arithmetic, and repeated parsing behavior), which we encode as structured hints for downstream reasoning. Next, in the field detection step, SEMPRE uses an LLM-based tree-of-thought search to propose candidate segmentations that are consistent with the collected taint/loop evidence. To address nested protocols, SEMPRE exploits the LLM's capability to recognize sub-payload boundaries and recursively applies the same detection procedure to any inferred nested regions, yielding an initial *hierarchical* message layout. Given this layout, our system performs semantic inference to predict the type and role of each field. The inferred semantics are then fed back into a subsequent detection round: high-confidence boundaries are frozen, and ambiguous regions are re-segmented under stronger semantic constraints. Then, SEMPRE alternates between field detection and semantic inference, using the output of each step to constrain the other until a fixed point is reached. If the process fails to converge within a preset budget, SEMPRE restarts with alternative hypotheses to escape poor local optima. Inspired by the Expectation–Maximization (EM) algorithm [21], we formalize this procedure as an iterative

optimization process and prove that each update is monotonic and that the procedure converges to a local optimum. The end result is a heuristic-free, effective PRE solution that improves both accuracy and stability compared to state-of-the-art techniques.

We implemented a prototype of SEMPRE and evaluated it against six state-of-the-art PRE tools. The results demonstrate that SEMPRE achieves the best overall performance across a diverse set of popular protocols. In field detection, SEMPRE attains the highest average F1 score of 0.76, demonstrating a 90%+ improvement with no other tools exceeding 0.4. In semantic inference, SEMPRE achieves an average F1 score of 0.72, substantially outperforming the 0.14 of BinPRE, the state-of-the-art PRE tool. Our further ablation study confirms and quantifies the contribution of each design component. In addition, we demonstrate SEMPRE's practicality for security tasks by using it to analyze the protocol of a real-world, widely distributed botnet malware sample.

Contributions. Our contributions are summarized as follows:

- We designed a fully automated PRE framework, which integrates fine-grained dynamic program analysis with a recursive LLM-driven reasoning procedure that performs *joint* optimizations over field detection and semantic inference.
- We formally proved that our algorithm converges to a local optimum.
- We implemented a prototype of SEMPRE and open-sourced our implementation [1] to facilitate future research in this field. We further evaluated its efficacy. Our experiments demonstrate that SEMPRE significantly outperforms prior PRE mechanisms and excels in analyzing real-world malware protocols (§IV).

II. Problem Statement & Motivation

A. Problem Statement

We formally define the PRE problem as follows. Let $\mathcal{M}$ denote the space of protocol messages. Each message $m \in \mathcal{M}$ is a finite byte sequence

$$m = (b_0, b_1, \ldots, b_{|m|-1}), \quad b_i \in \{0,1\}^8.$$

Field detection. A (flat) segmentation of m is an ordered list of K byte intervals

$$s = \big((l_1, r_1), (l_2, r_2), \ldots, (l_K, r_K)\big)$$

such that (i) each interval is contiguous with $0 \le l_k \le r_k \le |m|-1$, (ii) intervals are non-overlapping and ordered, i.e., $r_k + 1 = l_{k+1}$ for all $k \in \{1, \ldots, K-1\}$, and (iii) they cover the entire message, i.e., $l_1 = 0$ and $r_K = |m|-1$. Let $\mathcal{S}(m)$ denote the set of all valid segmentations of m. Field detection seeks a segmentation that maximizes a global structural objective:

$$f_{\text{det}}(m) = \arg\max_{s \in \mathcal{S}(m)} J(m, s),$$

where $J(m, s)$ scores structural plausibility (e.g., boundary quality and cross-field constraints).

Semantic inference. Let $\mathcal{Y}_{\text{common}}$ be a vocabulary of semantic labels (field types/functions), and let

$$\mathcal{Y} = \mathcal{Y}_{\text{common}} \cup \{\perp\},$$

where $\perp$ denotes "unknown / none of the above." Given a message m and a detected segmentation s with K fields, semantic inference assigns one label per field by maximizing a global semantic objective:

$$f_{\text{sem}}(m, s) = \arg\max_{y \in \mathcal{Y}^K} G(m, s, y),$$

where $y = (y_1, \ldots, y_K)$ and $G(m, s, y)$ evaluates semantic consistency across fields and with the observed bytes. For each field k, $y_k \in \mathcal{Y}_{\text{common}}$ indicates an assigned semantic type/function, while $y_k = \perp$ indicates that no suitable label is assigned.

In this study, we reuse $\mathcal{Y}_{\text{common}}$ defined in BinPRE [2] (i.e., five common types: static, integer, group, bytes, and string, and six common semantic functions: command, length, delim, checksum, filename, and aligned, detailed in Table XI) for fair comparisons in our evaluation. Note that adding new types or functions is straightforward and requires minimal configuration in SEMPRE, since it is built on an LLM that retains extensive knowledge of types and functions.

B. Motivation

To better understand the limitations of current PRE techniques, we examine several representative scenarios drawn from real-world network protocols and highlight the core challenges that existing tools struggle to address.

Inflexible Heuristics-Based Rules. Recent PRE mechanisms (e.g., BinPRE [2]) rely heavily on hand-crafted inference rules, typically designed by experts based on well-documented protocols and prior knowledge, to identify and label protocol fields. These rules inevitably introduce bias and limit the potential for generalization. For instance, Figure 2 shows a real-world TFTP message containing a filename field (Byte[2-17]). The file name here ("pxelinux.cfg/C0") includes a special slash character that is explicitly excluded by BinPRE's filename identification heuristic, but permitted by TFTP's specification [22]. Consequently, BinPRE fails to correctly recognize field semantics because of its rigid rules.

In contrast, SEMPRE avoids such brittle, protocol-specific heuristics by grounding an LLM's semantic hypotheses in dynamic execution evidence (e.g., tainting information and parsing behavior), enabling it to generalize to unseen but spec-compliant field values rather than rejecting them due to hard-coded pattern rules.

Ignorance of Semantic Relationships. Protocol fields rarely occur in isolation; instead, they are tied together by structural and semantic dependencies that govern correct parsing and interpretation. A common example is the *Length–Value (LV)* construct, in which a length field specifies the size of a subse-

[1] https://anonymous.4open.science/r/SemPRE-F9F8

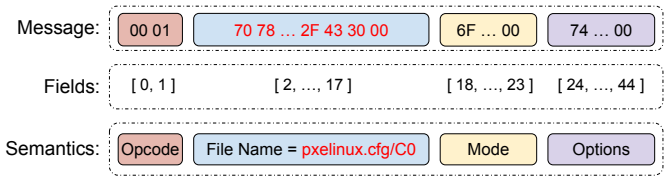


Fig. 2: Inflexible Heuristics-Based Rules

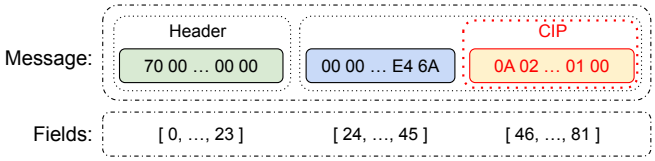


Fig. 4: Insufficient Handling of Nested Protocols

quent variable-length field, allowing the receiver to precisely delimit the payload. Figure 3 shows such dependencies in an S7 communication message [23]. In this example, byte[4] encodes a length that indicates the next 2 bytes form a payload segment. Likewise, byte[2–3] encodes the total message length, constraining the maximum extent of all subsequent fields. These length declarations create explicit cross-field constraints that, if modeled, can substantially reduce ambiguity in both field boundary detection and semantic labeling.

Existing PRE techniques do not reason about these inter-field dependencies, which makes them prone to cascading errors. For instance, a boundary mistake around a length field can distort the inferred layout of all downstream fields, and semantic mislabeling can remove key constraints needed for correct segmentation. For instance, BinPRE infers semantics at the byte (or local) level and labels byte[4] as a fixed-width numeric field without linking it to the following payload, and it even misinterprets byte[2–3] as two independent "Length" fields rather than a single two-byte length value. SEMPRE addresses this limitation by *jointly* optimizing field boundaries and semantic labels in an iterative loop, allowing inferred LV constraints (e.g., a length field governing a following payload) to validate and refine segmentation decisions rather than treating each field in isolation.

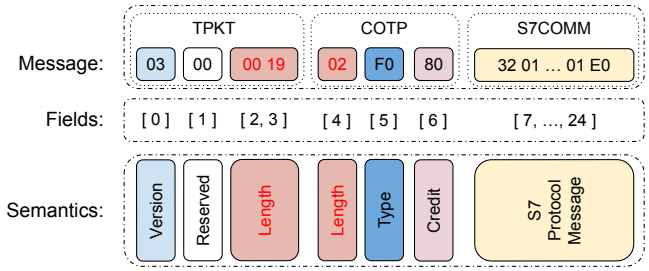


Fig. 3: Ignorance of Semantic Relationships

Insufficient Handling of Nested Protocols. Nested protocol structures are pervasive in real-world communication stacks, especially in industrial settings. A representative example is EtherNet/IP (EIP) [16], which commonly encapsulates Common Industrial Protocol (CIP) [17] payloads. In practice, a single EIP message may contain one or more embedded CIP messages, each with its own internal field layout and semantics. As shown in Figure 4, the example EIP message includes a CIP sub-payload spanning byte[46–81], accounting for nearly half of the message.

Current techniques implicitly assume a single, homogeneous message format. Hence, they either stop at the outer-layer header/payload boundary (leaving the embedded protocol unanalyzed) or flatten the entire byte stream into a single structure, which often leads to missegmentation and incorrect semantic labels for the inner protocol. SEMPRE explicitly targets nested protocols via *recursive* inference.

III. System Design and Implementation

A. Overview

As depicted in Figure 5, SEMPRE comprises three principal phases, namely pre-processing, field detection, and semantic inference, of which the latter two are coupled through a joint optimization process.

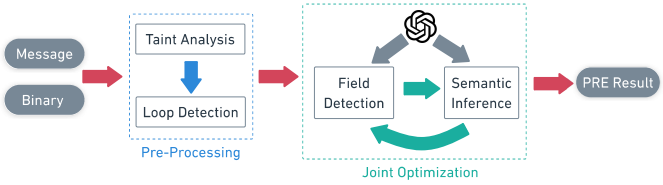


Fig. 5: SEMPRE Overview

Given a raw message from an undocumented protocol and its corresponding binary, preprocessing runs the binary on the message under dynamic taint analysis. We record per-byte dataflow and detect parsing loops along the executed path, exposing where the program repeatedly consumes input. These taint/loop cues reliably indicate variable-length arrays and length-prefixed regions, helping localize field boundaries and size dependencies. The taint map, loop inventory, and raw message are then used to construct an initial segmentation by aligning taint boundaries with loop patterns and other structural cues. Next, semantic inference assigns each segment a data type and functional role (e.g., length, command, string) using evidence extracted from the binary and statistical priors. High-confidence semantic predictions are fed back as constraints to refine segmentation (e.g., enforcing length–payload consistency). Field detection and semantic inference alternate until the joint objective stops improving; if needed, we restart with a different initialization up to a fixed budget. The final output is a stable, semantically annotated message layout. Notably, we operate on each message independently and avoid cross-message semantic alignment or majority-vote reconciliation. Without ground truth, global alignment can amplify noisy predictions and propagate errors across messages; instead,

SEMPRE emphasizes per-message internal consistency guided by execution evidence.

This process mirrors the workflow of a human analyst: an initial coarse segmentation and semantic assignment is iteratively refined by identifying high-confidence labels and using them to revise earlier boundary and type hypotheses, repeating until the message description becomes coherent and self-consistent. In SEMPRE, the analyst's evaluative role is delegated to an LLM. Recent studies [24], [25], [26], [27] show that state-of-the-art models (e.g., GPT-4) can perform self-critique and error correction when provided access to intermediate reasoning artifacts, and prior work [28], [29], [30] finds their judgments often track human preferences, making them effective automated evaluators in iterative pipelines [31]. Accordingly, after each iteration, the LLM reviews the current hypotheses, flags outputs it deems reliable, and feeds these constraints into the next cycle to progressively improve both segmentation and semantic interpretation.

B. Pre-processing

In the pre-processing stage, given a message and a binary that consumes the message, our system performs a byte-level dynamic taint analysis [32] that taints the given message as the data source and keeps track of how the binary processes the given message during the execution by recording how the binary's instructions access, propagate, and transform the tainted data from the input message. By doing so, SEMPRE can produce a set of taint information that contains details on which instructions read from, write to, or otherwise depend on each part of the input message during execution, which is essential for further segmenting the message into multiple fields. This information provides execution-level evidence for subsequent field segmentation. Since SEMPRE performs semantic reasoning on a per-message basis, its effectiveness does not primarily depend on the number of execution traces collected for the same message. To further facilitate field detection, we perform loop detection [33] to recognize loops during the execution and record the memory addresses of the loops in order to correlate repetitive processing patterns observed in the execution trace with the specific message bytes being handled. Identifying loops will highlight code that iterates over portions of the input, revealing variable-length fields or length-prefixed blocks. Such iteration patterns offer clear markers for field boundaries and size relationships, thereby sharpening the accuracy.

C. Field Detection

As depicted in Figure 6, our field detection takes as input the message, the taint and loop information from the pre-processing, and the inferred semantics from semantic inference (will be empty in the first iteration, more details in Section III-D), and eventually outputs the field detection result. To achieve this, SEMPRE leverages a novel prompting technique, Tree-of-Thoughts (ToT) [34], which enables LLMs to explore multiple reasoning paths through thoughts, leading to more accurate problem-solving. Specifically, ToT in field

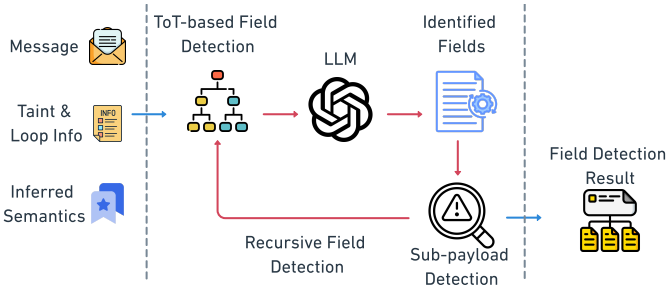


Fig. 6: Field Detection

detection enables the LLM to cross-validate multiple candidate segmentations against taint, loop, and semantic cues, thereby pruning inconsistent branches and curbing hallucinations while arriving at a globally coherent field layout, which effectively improves accuracy.

Tree-of-Thoughts (ToT) in Field Detection. ToT is a prompting framework that extends chain-of-thought prompting and self-consistency [35], [36]. Instead of committing to a single linear reasoning trace, ToT explores multiple candidate reasoning trajectories in parallel, organized as branches of a tree. Each node corresponds to a *thought*: a short textual intermediate step that is sufficiently concise to enable diverse generation, yet informative enough to support reliable evaluation. A typical ToT workflow comprises four steps: (1) *thought decomposition*, which defines the intermediate steps appropriate for the task; (2) *thought generation*, which proposes k candidate thoughts for the current state (e.g., via sampling); (3) *state evaluation*, which scores candidates by how much progress they make toward a solution and selects the most promising states to retain; and (4) *search*, which expands the tree using a strategy such as breadth-first or depth-first search, iterating until a satisfactory solution is found.

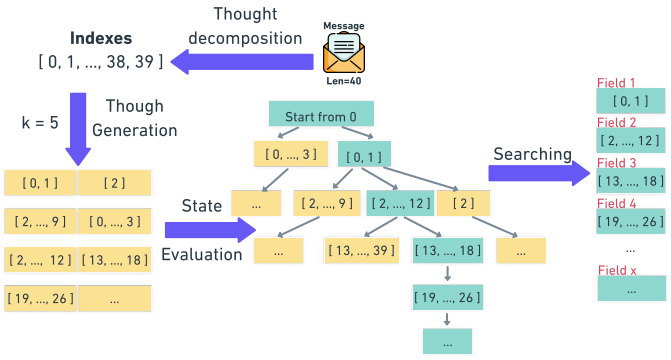


Fig. 7: Tree-of-Thoughts Process

To use ToT effectively in the context of SEMPRE, we instantiate it as a structured search procedure for message segmentation that explicitly captures semantic dependencies between adjacent fields. We treat *field boundaries* as the ToT search space: a ToT *state* represents a partial segmentation of the message prefix, and each expansion hypothesizes the next contiguous field starting at the first unassigned byte. Given

a raw protocol message of length L, the initial state spans indices $[0, 1, \ldots, L-1]$ and the search begins at index $i = 0$. Because protocol fields are contiguous and sequential, we map one "thought" to one complete field. Concretely, at position i, a thought proposes the next field as a contiguous byte range $[i, j]$ (with $j \geq i$). Selecting j uniquely determines the next start position $i \leftarrow j + 1$, so later boundary decisions are naturally conditioned on earlier ones.

In the thought generation phase, starting from the current index i, SEMPRE proposes up to k (we empirically set to 5) end positions j, yielding k alternative field hypotheses $[i, j]$. These candidates are not arbitrary: they are generated by jointly leveraging dynamic taint cues (e.g., grouping bytes consumed by the same semantic operation) and LLM-derived protocol cues (e.g., delimiter patterns, length constraints, and value formats). Figure 7 illustrates this process on a message of length $L = 40$ (indices $[0, \ldots, 39]$): at $i = 0$, the model may propose candidate boundaries such as $[0, 1]$ and $[0, \ldots, 3]$; after choosing $[0, 1]$, the next step begins at $i = 2$ and may generate candidates including $[2, 2]$, $[2, \ldots, 9]$, and $[2, \ldots, 12]$.

In the state evaluation phase, each candidate field induces a successor state (i.e., a longer prefix segmentation), which the LLM scores using a cost-based function, which reflects: (1) whether tainted bytes used in semantic operations are consistently covered by the proposed fields, (2) whether boundaries violate inferred structural/semantic constraints (e.g., length–payload consistency), and (3) whether the partial layout is plausible under common protocol patterns. The cost value C lies in [0, 1] for every path, with $C = 0$ denoting a perfectly consistent segmentation and $C = 1$ denoting one that violates every constraint on every field. SEMPRE retains only low-cost states, pruning implausible segmentations early while preserving diverse, high-quality boundary hypotheses.

Finally, in the search phase, SEMPRE performs a tree search over these states. Starting at $i = 0$, we apply depth-first search to enumerate complete segmentations and select the lowest-cost root-to-leaf path that reaches the end of the message. In Figure 7, the final field-detection result is the concatenation of the fields along this best path (i.e., Field 1: $[0, 1]$, Field 2: $[2, \ldots, 12]$, Field 3: $[13, \ldots, 18]$, Field 4: $[19, \ldots, 26]$).

Recursive Sub-payload Detection. We leverage the LLM to perform sub-payload detection, shown in Figure 6. When the LLM identifies a potential sub-payload region inside a protocol data unit (PDU), we *recursively* invoke the ToT-based field detection procedure described earlier on that sub-region, providing the same inputs (e.g., taint-flow and loop evidence, as well as any currently inferred semantics). This recursive process continues until the LLM reports that no further sub-payloads remain.

As illustrated in Algorithm 1, it takes as input a message M, the taint and loop information $Taint$ and $Loop$ from pre-processing, and high-confident semantic labels $SemLabels$ from semantic inference (which will be empty in the first iteration), and outputs $Fields$ as detection results. Specifically, we first create a field-detection prompt $Prompt_{field}$ using the given parameters (Ln. 2) and query the LLM to obtain $Response$, which contains the initial field set $Fields$. Next, we build a sub-payload detection prompt by combining contextual information $Log+Fields$ with sub-payload prompt template $P_{subpayload}$ (Ln.7) and query LLM to get $Response$ (Ln. 8), which provides updated fields of the message and an indicator showing whether additional sub-payload detection is required. Taint information in the context informs LLM which bytes are fixed, allowing it to reason only over untainted regions. If another iteration is needed, then we update Log and repeat; otherwise, the fields are the results.

Algorithm 1: Field Detection

```
1  FieldDetection(M, Taint, Loop, SemLabels);
   Input : M - A message of an unknown protocol;
           Taint - Taint information;
           Loop - Loop information;
           SemLabels - High-confident semantic labels;
           ToT - ToT parameters.
   Output: Fields - Detected fields of the message;
2  Prompt_field ← Create a field detection prompt using
     (Msg, Taint, Loop, SemLabels, ToT) ;
3  Response ← LLMQuery(Prompt_field) ;
4  Fields ← Responce;
5  Log ← Prompt_field;
6  while i ≤ MaxIterTimes do
7      Prompt_field ← Log + Fields + P_subpayload;
8      Response ← LLMQuery(Prompt_field);
9      Fields, SubPayloadFlag ← Response;
10     if SubPayloadFlag == True then
11         Log ← Prompt_field;
12         i++;
13         continue;
14     else
15         break;
16     end if
17 end while
18 return Fields
```

D. Semantic Inference

Following field detection, SEMPRE proceeds to semantic inference depicted in Figure 8. The inference stage is initialized with two resources: (i) a diverse exemplar set covering various protocol categories (e.g., binary, textual, or mixed messages), and (ii) the catalog of candidate type–function pairs defined by BinPRE [2]. Because a field's interpretation depends on neighboring fields and message-level constraints, semantic inference cannot be performed as an independent label assignment. For instance, a single field often admits several plausible type–function hypotheses whose validity is contingent on adjacent semantics (e.g., a putative length indicator dictates how the succeeding payload must be parsed).

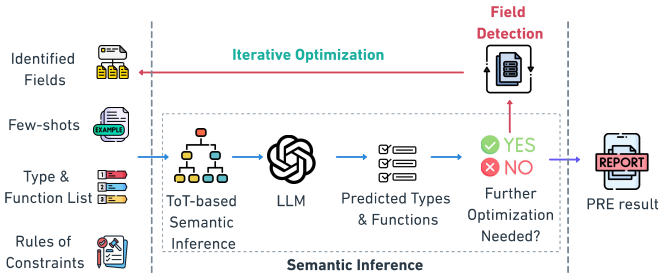


Fig. 8: Semantic Inference and Joint Optimization

To this end, we formulate semantic inference as a structured search and realize it with another Tree-of-Thoughts (ToT) prompting scheme. Each field produced by the field-detection phase constitutes one "thought" step: at that node, the LLM generates candidate type–function pairs by analyzing byte patterns, taint-/loop-derived behavior, and higher-level protocol cues. Successive steps branch to form alternative semantic paths over the message. Every path is cost-scored according to constraint consistency and corpus priors; a branch-and-bound mechanism then prunes incoherent paths, retaining only those with minimal cost. Exploration proceeds in sliding windows of 5 fields, iterating until all segments have been evaluated. During the process, the LLM also serves as an internal evaluator. Paths whose cost falls below a calibrated threshold are marked as high-confidence, and their labels are fixed as additional constraints for the next iteration. Through this iterative coupling of field detection and semantic inference, SEMPRE progressively converges on a globally coherent structural and semantic interpretation of the message. Complete prompt templates are included in Appendix A.

E. Joint Optimization

Based on our unique insight that field detection and semantic inference are strongly correlated and can complement each other, SEMPRE performs an iterative process between the two to jointly optimize results and reach a local optimum, as shown in Figure 8. This component is inspired by the Expectation–Maximization (EM) algorithm [21], which addresses problems involving partially observed data. In particular, after the initial inference round, SEMPRE has identified message fields and predicted their types and functions. Among these, some predictions can be marked as high-confidence (evaluated by the LLM) and serve as partially observed labels that guide the next iteration. This process is also inspired by how human analysts approach reverse engineering tasks. Analysts often begin with uncertain or approximate field interpretations, refine their understanding as more structural or semantic cues emerge, and use confident insights to correct earlier errors. SEMPRE mimics this behavior by using the LLM to evaluate its own intermediate outputs and determine whether further refinement is warranted. Through this feedback loop, SEMPRE incrementally improves its predictions, correcting earlier mistakes and reinforcing accurate interpretations. The refinement process continues until semantic convergence is achieved or no further improvements can be made.

Algorithm 2: Joint Optimization

```
1  Fields = {}, SemLabels = {};
2  while i ≤ MaxIterationTimes do
3      Fields ← FieldDetection(M, Taint, Loop, SemLabels);
4      SemLabels ← SemanticInference(M, FewShots, Fields);
5      Fields_h, SemLabels_h, NeedFurtherOp ← LLMQuery (Fields, SemLabels);
6      update (Type_h, Fn_h);
7      if NeedFurtherOp == True then
8          Fields = Fields_h;
9          SemLabels = SemLabels_h;
10         i++;
11         Go to the next iteration ;
12     else
13         return Fields, SemLabels;
14     end if
15 end while
16 if i > MaxIterationTimes then
17     Restart the whole process ;
18 end if
```

Optimization Algorithm. The optimization procedure is summarized in Algorithm 2. We initialize the field detection results, denoted by $Fields$, and the semantic inference results, denoted by $SemLabels$, as empty sets. The algorithm then proceeds iteratively through the following stages: (1) invoke Algorithm 1 to obtain a provisional set of fields (Ln.3); (2) apply few-shot exemplars, constraint rules, and the interim fields to infer semantic labels (Ln.4); and (3) query the LLM to select high-confidence predictions and determine whether additional joint optimization is needed (Ln.5). If further optimization is required, the high-confidence predictions are fixed, and the next iteration is executed (Ln.7–11). Otherwise, the algorithm returns $Fields$ and $SemLabels$ as the final outputs (Ln.13). Moreover, if the current solution has not converged, SEMPRE restarts the entire process and repeats the optimization until a predefined limit is reached (Ln.16–17).

F. Analysis of Iterative Refinement

SEMPRE alternates between field detection and semantic inference, using high-confidence predictions from one iteration as constraints for the next. Because both stages involve stochastic LLM inference, we do not claim unconditional convergence for arbitrary executions. Instead, we analyze the refinement process under a structural property of the PRE problem and two practical behavioral assumptions. The analysis characterizes when the iterative process is expected to stabilize.

State and Objective. Given a message B and its dynamic analysis information T, let S_t and L_t denote the field segmentation and semantic labels at iteration t, respectively. Let H_t denote the set of high-confidence predictions retained as constraints. We use $J(S, L \mid B, T)$ to denote the consistency of a candidate interpretation with runtime evidence and cross-field constraints, where a larger value indicates a more consistent interpretation.

Finite Candidate Space. For a finite message, the number of possible contiguous segmentations is finite. Since SEMPRE assigns each field a label from a finite semantic vocabulary, the joint candidate space of (S, L) is also finite. Likewise, H_t can contain only finitely many constraints. This property follows directly from the discrete formulation of field segmentation and semantic labeling.

Assumption 1: Reliable High-Confidence Constraints. Following prior findings on the reliability of high-confidence LLM predictions, we assume that predictions retained in H_t are sufficiently reliable to serve as constraints in subsequent iterations. In particular, we assume that fixing these predictions does not introduce erroneous constraints that prevent refinement toward an interpretation more consistent with the available runtime evidence. The retained constraints therefore accumulate as

$$H_t \subseteq H_{t+1}. \tag{1}$$

Assumption 2: Consistency-Preserving Refinement. We further assume that an accepted refinement does not decrease the consistency objective:

$$J(S_{t+1}, L_{t+1} \mid B, T) \geq J(S_t, L_t \mid B, T). \tag{2}$$

This condition captures SEMPRE's intended behavior: newly identified semantic constraints, such as a reliable length–payload relationship, reduce ambiguity in subsequent field detection, while refined field boundaries constrain semantic inference.

Under Eq. 2, the sequence

$$J_t = J(S_t, L_t \mid B, T)$$

is non-decreasing. Because the candidate space is finite, J_t cannot strictly increase indefinitely. Moreover, if an iteration does not improve J_t but adds new reliable constraints, H_t can strictly grow only finitely many times. Therefore, if SEMPRE continues refinement only when either

$$J_{t+1} > J_t \quad \text{or} \quad H_t \subset H_{t+1}, \tag{3}$$

The process eventually reaches a state in which neither condition holds. We call this a stable fixed point of the refinement procedure.

Practical Interpretation. The above result is conditional, not a formal guarantee of local or global optimality. In practice, an LLM may generate an inconsistent hypothesis or incorrectly mark a prediction as high-confidence, violating the behavioral assumptions above. We can observe behavior consistent with these assumptions across iterations by tracking the accumulation of high-confidence predictions and changes in the predicted field boundaries and semantic labels. SEMPRE also bounds the number of refinement iterations and restarts the process when it does not stabilize within the predefined budget.

Thus, the analysis characterizes when SEMPRE's refinement process reaches a stable fixed point, rather than claiming convergence to a local or global optimum.

IV. EVALUATION

We evaluate our system with respect to its general effectiveness and its efficacy in handling real-world cases. Specifically, we aim to conduct a series of experiments to answer the following research questions.

- **RQ1**: How effective is SEMPRE for field detection?
- **RQ2**: How effective is SEMPRE for semantic inference?
- **RQ3**: Can SEMPRE maintain its effectiveness without potential prior knowledge from the LLM or with a different backbone LLM?
- **RQ4**: Does each component of SEMPRE contribute uniquely in protocol reverse engineering?
- **RQ5**: Can SEMPRE effectively handle real-world malware protocols?

A. Experimental Setup

Baseline Techniques. To evaluate the effectiveness of SEMPRE, we compare its performance against six state-of-the-art PRE techniques, including both network-traffic-based and binary-analysis-based approaches. Specifically, we compare SEMPRE with PolyGlot [13], Tupni [14], AutoFormat [37], BinPRE [2], BinaryInferno [8], and DynPRE [10]. Note that we directly reuse the code released by the original authors for BinPRE, PolyGlot, AutoFormat, and Tupni, and re-implement BinaryInferno and DynPRE ourselves based on their papers since their code is unavailable.

Dataset. We constructed the dataset as follows.

- *Protocols.* We collect three protocol datasets for evaluation: **D1**) the dataset in BinPRE [2], which includes eight widely used protocols covering textual, binary, and mixed formats across the industrial control and network communication domains; **D2**) a real-world undocumented dataset from GitHub, which contains three real-world undocumented protocols (KTP [38], CCN [39] and Xfer [40]); **D3**) we use Mirai [41], which is a botnet malware protocol, as a case study. Further information is listed in Table XV.
- *Messages.* Detailed in Table I in Appendix, we collect two message datasets: (1) open-source message datasets and (2) datasets built by ourselves when open-source datasets are unavailable. Concretely, we build the datasets for DNP3.0, HTTP, Mirai, KTP, CCN, and Xfer using their clients and the `tcpdump` tool. The other messages are from open-source datasets. The number of messages varies across protocols, as the sources of messages differ by protocol.

Additionally, the network-based PRE methods, BinaryInferno and DynPRE, are designed to handle small input messages. Please note: although BinPRE reports results, its public repository does not release the corresponding messages (except Modbus), so we evaluated them using our own captured ones.

TABLE I: Groud Truth Details.

Dataset	Protocol	# Messages	# Fields	# Types	# Fn.s	R.T.	R.F.
D1	Modbus	50	378	378	100	1	0.26
	S7comm	41	798	526	372	0.66	0.47
	Ethernet/IP	50	2802	2346	1046	0.84	0.37
	DNP3	53	692	452	212	0.65	0.31
	DNS	50	450	450	206	1	0.46
	FTP	38	168	168	137	1	0.82
	TFTP	16	112	96	32	0.86	0.29
	HTTP	62	1414	1166	248	0.82	0.18
D2	KTP	9	54	54	9	1	0.17
	CCN	8	64	64	24	1	0.38
	Xfer	9	90	84	24	0.93	0.27
D3	Mirai	10	127	127	45	1	0.35
D4	Ethernet/IP	5					
	BAC	5				1	
	OPC	5	80	80	26	1	0.33
Summary		396	7149	5911	2455	0.82	0.35

R.T. - ratio of labeled types; R.F. - ratio of labeled functions

Ground Truth Collection. The primary source of ground-truth annotations is the official protocol documentation. For Mirai, whose protocol lacks a formal specification, we instead derive the annotations through a manual audit of the publicly available source code. The resulting corpus records, for every message, the byte-level field boundaries together with a single type and a single function label for each field. All annotations are cross-validated against Wireshark packet dissections, yielding a data set we regard as error-free. Type and function labels are absent for a small subset of fields when the available taxonomy proves insufficient or when discrepancies arise between the written specification and the Wireshark interpretation.

Evaluation Metrics. Field detection produces a set of byte offsets that mark the field boundaries. Each predicted offset is classified as either a true positive or a false positive with respect to the reference annotations, enabling the computation of precision (the proportion of predicted boundaries that are correct), recall (the proportion of actual boundaries that are recovered), and their harmonic mean, the F1 score. Semantic inference, likewise, is evaluated through these three measures: although the task is nominally multi-class, each type or function assignment can be viewed as a binary decision (correct vs. incorrect) relative to the ground truth. Accordingly, we aggregate the per-field outcomes to obtain precision, recall, and F1 for type labels and function labels. Throughout the study, these three metrics (precision, recall, and F1) serve as the basis for our performance assessment.

Please note that BinPRE uses offset-level metrics for field detection, considering all bytes in a given message, but this is a flaw because it significantly reduces the impact of incorrect boundaries. For example, BinPRE outputs [-1,2,3,5,6] (5 offsets but 4 fields) against a ground truth [-1,1,5,6]. Based on BinPRE's calculation, 3 of 5 offsets match (-1, 5, 6), giving precision 0.6 even though only 1 field is correctly identified (5, 6). Thus, our metrics directly work at the field level. We check whether predicted field boundaries are correct, which is simpler and more direct than BinPRE.

B. RQ1: Effectiveness on Field Detection

To evaluate field detection, we compute precision, recall, and F1 score for SEMPRE on each protocol and aggregate the results across all messages. We compare against baseline methods using the D1 dataset, which includes eight widely used protocols spanning all three protocol categories. Overall, as shown in Figure 9, SEMPRE substantially outperforms all baselines. Compared to the strongest competitor, BinPRE, SEMPRE achieves a whopping 95% improvement (0.76 v.s. 0.39). Averaged across all baselines, the F1 gain is 0.45, and remains 0.41 when excluding the weakest baseline, BinaryInferno. In absolute terms, SEMPRE correctly identifies 4,822 of 6,465 ground-truth fields, exceeding the runner-up by 933 correct detections.

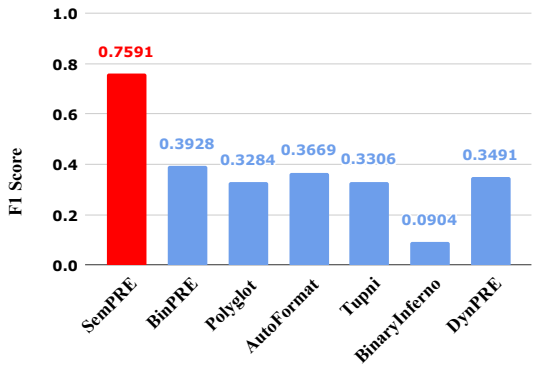


Fig. 9: Average F1 Score in Field Detection

For individual protocols, SEMPRE outperforms all baselines in nearly all cases (Table II), with two minor exceptions. For Modbus, BinPRE achieves a perfect F1 score (1.00), while SEMPRE attains 0.99, a negligible difference. For DNP3, AutoFormat achieves higher precision due to protocol-specific constraints. DNP3 produces the longest taint- and loop-analysis traces because its implementation validates nearly every byte, which limits SEMPRE to using less than 10% of the analysis data under OpenAI API input-length constraints. AutoFormat, by contrast, can leverage the full analysis. Even so, SEMPRE attains an F1 score of approximately 0.80 (about 90% of AutoFormat's performance), demonstrating strong robustness despite the reduced data.

Answer to RQ1: SEMPRE identifies fields with a solid F1 score of 0.76 on all protocol messages and can beat all state-of-the-art techniques.

TABLE II: Field Detection Results.

Protocols	SEMPRE			BinPRE			Polyglot			AutoFormat			Tupni			Binaryinferno			DynPRE		
	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1
Modbus	0.98	1.00	0.99	0.99	1.00	**1.00**	0.60	0.98	0.75	0.85	1.00	0.92	0.60	0.98	0.75	0.25	0.15	0.19	0.81	0.37	0.51
S7comm	0.81	0.81	**0.81**	0.82	0.02	0.05	0.47	0.16	0.23	0.43	0.13	0.19	0.47	0.16	0.23	0.00	0.00	0.00	0.84[1]	0.51[1]	0.64[1]
Ethernet/IP	0.86	0.75	**0.80**	1.00	0.15	0.25	0.36	0.61	0.45	0.37	0.60	0.46	0.36	0.61	0.45	0.03	0.01	0.01	0.86	0.11	0.19
DNP3	0.84	0.76	0.80	0.88	0.58	0.70	0.59	1.00	0.74	0.83	1.00	**0.91**	0.69	1.00	0.81	0.98	0.41	0.58	N/A	N/A	N/A
DNS	0.56	1.00	**0.72**	0.39	0.97	0.56	0.23	1.00	0.38	0.22	0.91	0.36	0.23	1.00	0.38	0.00	0.00	0.00	0.21[2]	0.80[2]	0.34[2]
FTP	0.57	0.79	**0.66**	0.36	0.38	0.37	0.20	0.58	0.30	0.17	0.44	0.25	0.20	0.58	0.30	0.00	0.00	0.00	N/A	N/A	N/A
TFTP	1.00	1.00	**1.00**	0.46	0.66	0.54	0.14	1.00	0.24	0.11	0.83	0.20	0.14	1.00	0.24	0.50	0.17	0.25	N/A	N/A	N/A
HTTP	0.63	0.57	**0.60**	0.27	0.36	0.31	0.07	0.40	0.12	0.10	0.40	0.16	0.07	0.40	0.12	0.02	0.00	0.00	N/A	N/A	N/A
Average	0.77	0.75	**0.76**	0.49	0.33	0.39	0.23	0.60	0.33	0.27	0.58	0.37	0.23	0.60	0.33	0.28	0.05	0.09	0.63	0.24	0.35

1 - / 2 - DynPRE only accepts part of the messages
1 - Counting based on the first 31 messages; 2 - Counting based on the first 16 messages.

C. RQ2: Effectiveness on Semantic Inference

To evaluate semantic inference, we assess type inference and function inference separately, comparing only against baselines that support the corresponding task. For type inference, we compare exclusively with BinPRE, the only baseline that predicts field types, using the same type categories for fairness. For function inference, we include all baselines except AutoFormat, which does not perform semantic labeling. Overall, SEMPRE correctly infers 3,441 of 5,582 reference types and 1,431 of 2,353 reference functions.

For type inference (Table III), SEMPRE consistently outperforms BinPRE across all evaluated protocols. Its F1 score on TFTP is comparatively lower (0.3542), but it still exceeds all baselines. This gap is primarily caused by *semantic ambiguity*: several TFTP fields are encoded as byte strings, yet they semantically represent numeric parameters or composite values, making type assignment difficult even for experienced analysts. BinPRE's weak results on S7comm, FTP, and HTTP largely stem from inaccurate field detection: when boundaries are wrong, semantic inference is inherently inaccurate. While BinPRE reaches an F1 of 0.70 on DNP3, it often assigns multiple type labels to the same field. We count such multi-label outputs as incorrect (even if one label matches the ground truth), because this behavior amounts to guessing and can artificially inflate precision (by up to 20%) without providing genuine semantic understanding.

TABLE III: Type Inference Results.

Protocols	SEMPRE			BinPRE		
	Precision	Recall	F1	Precision	Recall	F1
Modbus	0.82	0.79	**0.81**	0.41	0.40	0.41
S7comm	0.83	0.78	**0.80**	0.00	0.00	0.00
Ethernet/IP	0.90	0.60	**0.72**	0.33	0.04	0.08
DNP3	0.92	0.75	**0.82**	0.00	0.00	0.00
DNS	0.75	0.67	**0.71**	0.58	0.42	0.49
FTP	0.95	0.58	**0.72**	0.00	0.00	0.00
TFTP	0.35	0.35	**0.35**	1.00	0.16	0.27
HTTP	0.90	0.48	**0.62**	0.00	0.00	0.00
Average	0.86	0.62	**0.72**	0.45	0.08	0.14

For function inference (Table XIX in Appendix), SEMPRE again outperforms all baselines on every protocol, yielding a 90% improvement over the runner-up (BinPRE). Performance on S7comm, EIP, and HTTP is comparatively lower (0.64, 0.67, and 0.70, respectively), which we attribute primarily to boundary errors introduced during field detection. Notably, the precision for EIP and HTTP is close to 1.00, indicating that when field boundaries are correctly identified, the inferred function labels are highly reliable. S7Comm is particularly challenging because messages over TCP/IP are encapsulated by multiple headers, creating a deeply nested structure in which semantic errors at outer layers can propagate to inner sub-payloads.

> *Answer to RQ2:* SEMPRE identifies both field type and function with high accuracy, significantly outperforming all the state-of-the-art PRE techniques.

D. RQ3: Robustness and Generalization

SEMPRE uses GPT-o3 model as its backbone LLM, which raises the concern that the model may have been exposed to protocol specifications during pre-training, potentially introducing bias into our evaluation. To decouple SEMPRE's performance from any latent prior knowledge, we construct a *zero-prior-knowledge* setting. Concretely, our prompt explicitly forbids the use of any background knowledge about Modbus, S7Comm, Ethernet/IP, DNP3, DNS, FTP, TFTP, and HTTP, and requires field extraction to be derived solely from the provided message content and auxiliary traces. We validate that this constraint is effective by observing a clear performance degradation when running vanilla GPT-o3 under the zero-knowledge prompt (Table V). We then evaluate three configurations on the D1 dataset: SEMPRE, SEMPRE-zero, and BinPRE. As shown in Table IV, SEMPRE indeed benefits from background knowledge. However, even without it, SEMPRE-zero still significantly outperforms BinPRE on most protocols. The only exception is DNP3, where API input-length limits force us to truncate the taint context, reducing the amount of usable dynamic evidence and thereby weakening the method's effectiveness.

TABLE IV: Effectiveness w/wo Prior Knowledge.

Protocols	SEMPRE			SEMPRE-zero[1]			BinPRE		
	Field	Type	Fn.	Fields	Type	Fn.	Fields	Type	Fn.
Modbus	0.99	**0.81**	**1.00**	0.99	0.73	0.99	**1.00**	0.41	**1.00**
S7comm	0.81	**0.80**	0.64	**0.82**	0.59	0.35	0.05	0.00	0.00
Ethernet/IP	**0.80**	**0.69**	**1.00**	0.62	0.62	0.99	0.25	0.22	1.00
DNP3	**0.80**	0.82	0.90	0.64	0.34	0.31	0.70	0.00	0.86
DNS	**0.72**	**0.71**	**0.93**	0.65	0.55	0.72	0.56	0.49	0.75
FTP	**0.66**	**0.72**	**0.77**	0.63	0.68	0.74	0.37	0.00	0.00
TFTP	**1.00**	0.35	**1.00**	**1.00**	**0.41**	**1.00**	0.54	0.27	0.64
HTTP	0.60	**0.62**	0.70	**0.85**	0.55	0.54	0.31	0.00	0.00
Average	**0.76**	**0.72**	**0.74**	0.72	0.45	0.43	0.39	0.14	0.35

1 - SEMPRE-zero: SEMPRE in the zero-prior-knowledge setting.

We further evaluate on undocumented protocols (D2 dataset), whose specifications are unlikely to be memorized by LLMs. As shown in Table XVIII in Appendix, SEMPRE consistently outperforms both BinPRE and vanilla GPT-o3. Although SEMPRE and BinPRE rely on the same taint information, BinPRE groups bytes using the Needleman–Wunsch algorithm, which lacks instruction-level semantic awareness and often produces incorrect merges. In contrast, SEMPRE leverages LLM-guided consistency between field detection and semantic inference to interpret taint outputs and delineate boundaries more accurately. An exception applies to KTP, where performance is comparable between SEMPRE and vanilla GPT-o3, because half of the fields are firmly associated with the sending end, while the other fields work for the receiving end, and taint information affects only fields related to the sending end.

Finally, we replace GPT-o3 with DeepSeek-v3.1-terminus to examine whether SEMPRE's effectiveness changes substantially under a different backbone LLM. The results remain comparable and consistently outperform BinPRE (Table XVII), suggesting that SEMPRE's gains are not tied to any single model, but stem primarily from its overall design.

TABLE V: GPT-o3 Performance.

Protocols	w Knowledge			w/o Knowledge		
	Fields	Types	Fn.	Fields	Types	Fn.
Modbus	**0.67**	**0.63**	0.45	0.62	0.62	**0.57**
S7comm	**0.80**	**0.77**	**0.40**	0.18	0.19	0.06
Ethernet/IP	**0.38**	**0.38**	**0.12**	0.32	0.23	0.06
DNP3	0.27	0.36	0.27	**0.36**	**0.54**	**0.33**
DNS	0.70	0.84	0.39	**0.87**	**0.89**	**0.51**
FTP	**0.48**	**0.79**	**0.85**	0.38	0.67	0.74
TFTP	**1.00**	**0.69**	**1.00**	0.77	0.62	0.91
HTTP	0.48	0.64	0.21	**0.50**	**0.66**	**0.24**
Average	**0.53**	**0.58**	**0.34**	0.44	0.46	0.29

Answer to RQ3: SEMPRE maintains strong performance without prior LLM knowledge or with a different backbone LLM, consistently outperforming SOTA techniques.

E. RQ4: Ablation Study

In this study, we explore the effectiveness of two core components of SEMPRE: recursive sub-payload detection (Figure 6) and iterative optimization (Figure 8). To conduct this ablation study cost-efficiently, we use a small-scale dataset derived from RQ1 by selecting the first 10 messages from each protocol. We evaluate the following configurations of SEMPRE:

- **SEMPRE-norec-noop**: remove both the recursive sub-payload detection and the iterative optimization
- **SEMPRE-noop**: remove the iterative optimization
- **SEMPRE-ToT**: remove the ToT
- **SEMPRE-norec**: remove the recursive sub-payload detection
- **SEMPRE**: full-fledged SEMPRE

Recursive sub-payload detection addresses the challenge of field detection in the presence of nested payloads. As shown in Table VII in Appendix, adding this component increases the average F1 score from 0.58 to 0.76 compared to SEMPRE-norec-noop. Although DNS sees a decrease (1.00 to 0.77), substantial gains are achieved for Ethernet/IP ($0.37 \rightarrow 0.71$) and HTTP ($0.39 \rightarrow 0.62$). A similar performance shift between SEMPRE-norec-noop and SEMPRE further confirms the effectiveness of recursive detection.

TABLE VI: Ablation Study Results - Semantic Inference.

Protocols	**SEMPRE-norec-noop**		**SEMPRE-noop**		**SEMPRE**	
	Type F1	Fn. F1	Type F1	Fn. F1	Type F1	Fn. F1
Modbus	**0.79**	**1.00**	0.78	**1.00**	0.78	**1.00**
S7comm	0.68	0.53	0.86	0.70	**0.91**	**0.77**
Ethernet/IP	0.28	0.25	0.31	0.24	**0.47**	**0.62**
DNP3	0.75	0.82	**0.84**	**0.97**	0.80	0.90
DNS	0.71	0.87	**0.72**	**1.00**	0.70	0.96
FTP	0.68	0.71	**0.68**	**0.71**	0.67	**0.71**
TFTP	0.50	0.97	**0.40**	0.95	**0.40**	**1.00**
HTTP	0.31	0.18	0.65	0.49	**0.68**	**0.71**
Average	0.48	0.51	0.56	0.58	**0.62**	**0.74**

Improved field detection should also enhance semantic inference, as intended. SEMPRE-noop outperforms SEMPRE-norec-noop on semantic tasks, raising average F1 $0.48 \rightarrow 0.56$, for type inference and $0.51 \rightarrow 0.58$ for function inference (Table VI). Adding iterative optimization further boosts performance, raising F1 scores to 0.62 for type inference and 0.74 for function inference. While some protocols show minor drops, the gains on Ethernet/IP and HTTP are substantial, as shown in Table VI.

From these results, we observe that recursive sub-payload detection is effective: the total correct fields are 176 without sub-payload detection (vs. 483 in SemPRE) for the nested

TABLE VII: Ablation Study Results - Field Detection.

Protocols	SEMPRE-norec-noop			SEMPRE-noop			SEMPRE		
	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1
Modbus	1.00	1.00	**1.00**	1.00	1.00	**1.00**	1.00	0.84	0.91
S7comm	0.98	0.69	0.81	0.90	0.95	**0.93**	0.89	0.94	0.92
Ethernet/IP	0.96	0.23	0.37	0.81	0.63	0.71	0.87	0.61	**0.72**
DNP3	0.95	0.58	0.72	0.91	0.94	**0.92**	0.90	0.78	0.84
DNS	1.00	1.00	1.00	0.63	1.00	**0.77**	0.58	1.00	0.73
FTP	0.34	0.78	0.47	0.34	0.78	**0.47**	0.23	0.53	0.32
TFTP	1.00	1.00	1.00	0.91	1.00	0.95	1.00	1.00	**1.00**
HTTP	0.53	0.31	0.39	0.61	0.62	0.62	0.81	0.68	**0.74**
Average	0.84	0.45	0.58	0.77	0.75	**0.76**	0.81	0.72	**0.76**

TABLE VIII: Ablation Study Results - Sub-payload Detection.

Protocols	SemPRE-norec			SemPRE			BinPRE		
	bnd F1	type F1	fn F1	bnd F1	type F1	fn F1	bnd F1	type F1	fn F1
Ethernet/IP	0.55	0.12	0.13	**0.72**	**0.47**	**0.62**	0.19	0.00	0.13
BAC	**0.92**	**0.80**	**0.96**	0.84	**0.80**	0.90	0.79	0.21	0.15
OPC	**0.78**	**0.72**	0.95	0.73	0.70	**0.96**	0.36	0.00	0.00
Avg.	0.71	0.50	0.56	**0.76**	**0.62**	**0.74**	0.29	0.06	0.12

protocol Ethernet/IP. To strengthen the component's contribution, we build a small nested dataset $D4$ and evaluate SEMPRE-norec and SEMPRE on that. The results are shown in Table VIII.

SEMPRE-ToT degrades F1 from 0.76/0.62/0.74 to 0.71/0.50/0.56 (Table IX, compared to SEMPRE, indicating that ToT alone cannot improve performance but plays a key role in SEMPRE.

Overall, these results demonstrate that both recursive sub-payload detection and iterative optimization significantly enhance SEMPRE 's ability to extract fields and infer semantics.

Answer to RQ4: SEMPRE 's recursive sub-payload detection and iterative optimization are essential for high performance.

F. RQ5: Case Study

Finally, we evaluate SEMPRE's efficacy using a case study on a variant of the Mirai botnet [41], a well-known malware family that targets Internet-of-Things (IoT) devices and poses a persistent threat in the cybersecurity landscape. We simulate a realistic analysis scenario in which only the Mirai binary and captured network traffic are available. We analyze 10 messages exchanged between the command-and-control (C&C) server and an infected device, reflecting an early-stage reverse-engineering setting in which no protocol documentation is assumed. We evaluate both SEMPRE and BinPRE against a manually constructed ground truth derived from the source code. Across the ten messages, the ground truth contains 117 fields, 127 type instances, and 45 function labels.

BinPRE assumes that a program consumes each message buffer in a single pass. The Mirai binary, however, reuses the same buffer and parses it a second time, breaking BinPRE's offset arithmetic and leading to almost zero accuracy. In contrast, although SEMPRE and BinPRE rely on the same taint outputs, SEMPRE does not impose a single-pass assumption. Instead, it uses LLM-based reasoning over the collected evidence and can therefore accommodate repeated buffer accesses. To isolate this effect, we run an additional experiment in which both tools are explicitly informed that the Mirai binary traverses the message buffer twice. With this hint, BinPRE achieves an F1 score of 0.55 for field detection. By comparison, SEMPRE attains a stable 0.83 F1 for field detection with or without the hint. At the per-message level, SEMPRE recovers perfectly accurate boundaries for 9 out of 10 messages, whereas BinPRE does so for only 2. The remaining gap largely stems from BinPRE's dependence on the Needleman–Wunsch (NW) algorithm for aligning instruction sequences, as it captures primarily instruction-level correspondences and often misses deeper semantic relations needed to correctly merge and split tainted regions.

TABLE IX: Ablation Study Results - ToT Affect.

Protocols	SemPRE-ToT			SemPRE		
	bnd F1	type F1	fn F1	bnd F1	type F1	fn F1
modbus	**1.00**	0.78	**1.00**	0.91	0.78	**1.00**
s7	0.89	0.85	0.71	**0.92**	**0.91**	**0.77**
eip	0.55	0.12	0.13	**0.72**	**0.47**	**0.62**
dnp3	**0.92**	**0.80**	**0.96**	0.84	**0.80**	0.90
dns	**0.78**	**0.72**	0.95	0.73	0.70	**0.96**
ftp	**0.45**	0.61	0.67	0.32	**0.67**	**0.71**
tftp	**1.00**	**0.40**	**1.00**	**1.00**	**0.40**	**1.00**
http	0.72	0.66	0.62	**0.74**	**0.68**	**0.71**
Avg.	0.71	0.50	0.56	**0.76**	**0.62**	**0.74**

TABLE X: Case Study - F1 Scores.

	Field	Type	Function
SEMPRE	**0.83**	**0.69**	**0.74**
BinPRE	0.55	0.27	0.19

Answer to RQ5: The SEMPRE can effectively support real-world security analysis, particularly in scenarios where protocols are undocumented and malicious.

V. RELATED WORK

A. Protocol Reverse Engineering

Automated protocol reverse engineering (PRE) techniques broadly fall into two categories: *network-traffic-based* and *binary-analysis-based* approaches. Traffic-based systems infer message layouts by identifying recurring byte patterns across packet traces, whereas binary-analysis methods reconstruct formats and semantics by examining how a program processes its inputs. Early binary-analysis tools (e.g., Polyglot [13], AutoFormat [37], and Tupni [14]) use taint tracking and execution context analysis to extract field boundaries but provide limited semantic information. In parallel, traffic-based methods have evolved from largely heuristic systems such as NEMESYS [9]

to more structured approaches like NetPlier [11], which leverage sequence alignment and probabilistic inference to identify type-indicating fields and reconstruct protocol state machines. Recent work has improved both semantic depth and robustness. BinaryInferno [8] combines multiple heuristics in an entropy-guided ensemble to improve precision and completeness, while DynPRE [10] adopts an active-learning strategy that generates probe messages to explore protocol states and reveal richer semantics. BinPRE [2] represents the state of the art in binary-analysis-based PRE. It aligns instruction sequences, employs atomic field detectors for common data-handling idioms, and iteratively clusters and refines candidate fields. Evaluated on eight real-world protocols, BinPRE surpasses prior systems, including Polyglot, AutoFormat, Tupni, BinaryInferno, and DynPRE, in both precision and recall.

B. LLM Reasoning Techniques

Early progress in LLM reasoning stemmed solely from prompt engineering. Chain-of-Thought [36] (CoT) prompting revealed that inserting a few worked examples elicits multi-step reasoning, boosting accuracy on math and commonsense tasks. Self-Consistency [35] improves CoT by sampling diverse reasoning paths and selecting the majority answer, yielding double-digit gains across several benchmarks. Active-Prompt [42] further adapts the exemplar set on-the-fly, choosing those that best suit a new task. Despite these advances, all three methods rely on a single linear trace of text tokens and quickly degrade when the input interleaves structured bytes with noisy instrumentation—precisely the scenario in protocol reverse engineering (PRE).

To avoid the myopia of linear CoT, Tree-of-Thoughts [34] (ToT) treats reasoning as an explicit tree search, allowing the model to branch, backtrack, and evaluate partial solutions before committing; GPT-4 accuracy on the "24 Game" rises from 4% to 74% under this scheme. Yet ToT still assumes that each "thought" is a clean natural-language fragment, leaving open how to segment—and reason over—raw byte streams mixed with taint-trace noise. SEMPRE tackles exactly this gap by anchoring tree exploration in execution evidence.

LLMs can also grade and revise their own work. A recent study [43] shows that introducing a reflection loop—generate, critique, then refine—mainly lifts performance on multiple-choice QA and planning benchmarks. SEMPRE borrows this idea but grounds the critique in concrete taint maps and loop inventories, rather than in free-form textual judgment.

VI. DISCUSSION

LLM Limitations. Despite their impressive performance across a wide range of tasks, large language models (LLMs) are subject to several important limitations, particularly in terms of reliability and factual accuracy. One of the most critical issues is hallucination, where the model generates fluent but factually incorrect or fabricated outputs. Studies have shown that LLMs may confidently generate incorrect or hallucinatory answers [44], [45], produce inaccurate or fabricated sources [46], and exhibit reduced factual reliability when the required knowledge is insufficiently represented in their pre-training data [47], [48]. Furthermore, LLMs often lack grounding in external knowledge and can struggle with consistency across long or complex reasoning chains [31]. Although we apply Tree-of-Thoughts and iterative refinement to improve the LLM's effectiveness, the LLM's limitations are hard to eliminate entirely.

Byte-Level v.s Bit-Level Semantics. Our current implementation supports only six semantic function labels for fair comparison with BinPRE, far fewer than those found in real-world protocols. In addition, SEMPRE performs semantic annotation at *byte* granularity. This design follows directly from our reuse of BinPRE's taint-analysis backend, which tracks data dependencies at the byte level. While byte-level tainting integrates cleanly with dynamic program analysis and remains efficient in practice, it limits SEMPRE's ability to recover finer-grained semantics. This limitation is particularly salient for embedded and low-level protocols, where meaning is often encoded at the *bit* level rather than the *byte* level. For example, in the Controller Area Network (CAN) protocol, individual bits within a byte frequently represent distinct flags or sensor states (e.g., fault indicators and enable/disable toggles) [49]. Such semantics cannot be fully captured with byte-level inference alone, which constrains SEMPRE's precision in these settings. Extending SEMPRE to support bit-level taint tracking, bit-level semantic inference, and a richer taxonomy of semantic functions is an important direction for future work.

Robustness to Adversarial Settings. Obfuscation techniques that randomize field access order or copy data into temporary scratch buffers can blur taint signals and defeat ToT search. This problem involves defeating obfuscation, which is fundamentally orthogonal to the protocol reverse engineering task. We will leave this for future research.

VII. CONCLUSION

Protocol reverse engineering (PRE) reconstructs message structures and field semantics, yet existing tools rely on fragile heuristics and struggle with nested payloads and semantic dependencies. We present SEMPRE, a fully automated PRE framework that couples fine-grained dynamic analysis with a recursive LLM reasoning loop. Using taint-flow and loop traces, it segments messages, detects nested payloads, and iteratively refines structure and semantics through a monotonic EM-style process. On eight mainstream protocols, SEMPRE attains an F1 score of 0.76 for field detection—over twice the prior best—and raises semantic-inference accuracy to 0.72. It also proves effective on Mirai botnet traffic, accurately recovering undocumented fields and semantics. These results show that LLM-based reasoning can replace brittle heuristics and set a new state of the art in PRE.

APPENDIX

ETHICAL CONSIDERATIONS

Stakeholder Identification and Impacts. The research process and outcomes affect multiple stakeholders:

Research Team. Benefits from academic advancement and technical validation; bears responsibility for guiding ethical use.

Security Researchers/Analysts. Gains a high-precision tool for vulnerability discovery, malware analysis, and IDS signature generation, improving work efficiency.

Enterprises/Organizations. Can leverage SEMPRE to identify protocol vulnerabilities, but faces potential risks of legitimate protocol reverse-engineering by third parties.

General Users. Indirectly benefits from enhanced system security but may face privacy/property risks if the tool is misused.

Open Source Community. Obtains open-source resources to advance PRE research, promoting field development.

Security Community and Society

Beneficence. SEMPRE addresses limitations of heuristic-based PRE tools, doubling field detection F1 score (0.76 vs. ¡0.40) and improving semantic inference accuracy (0.72 vs. 0.14), advancing network security defense capabilities.

Respect for Persons. All experimental datasets are from open-source repositories or legally captured traffic, with sensitive information (IP addresses, device IDs) anonymized to protect user privacy.

Justice. The SEMPRE prototype is open-sourced to ensure fair access to technical resources, with transparent experimental design for community verification.

Respect for Law and Public Interest. The research and publication comply with network security regulations; we explicitly oppose misuse of the tool for illegal activities.

Potential Harms Analysis. We acknowledge that the protocols may create negative impacts if used by illicit actors to reveal private protocols. For example, an attacker may reconstruct the syntactic and semantic information of a private protocol using SEMPRE and then send spurious messages to attack normal users.

Ethical Rationale for Research and Publication

Research Justification. Traditional PRE tools rely on brittle heuristics, failing to handle nested protocols and semantic dependencies. SEMPRE fills this technical gap with LLM-driven joint optimization, critical for improving security defense.

Publication Justification. Open-sourcing and publishing enable academic exchange, allowing the community to refine the tool. Ethical guidelines and positive application scenario descriptions minimize misuse risks, ensuring alignment with public interest.

OPEN SCIENCE

Our code is available at https://anonymous.4open.science/r/SemPRE-F9F8.

REFERENCES

[1] J. Narayan, S. K. Shukla, and T. C. Clancy, "A survey of automatic protocol reverse engineering tools," *ACM Computing Surveys (CSUR)*, vol. 48, no. 3, pp. 1–26, 2015.

[2] J. Jiang, X. Zhang, C. Wan, H. Chen, H. Sun, and T. Su, "Binpre: Enhancing field inference in binary analysis based protocol reverse engineering," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 3689–3703.

[3] "Modbus protocol," https://www.modbustools.com/modbus.html, 2025.

[4] J. Wang, Q. Liu, C. Liu, and J. Yin, "Gan-based adversarial patch for malware c2 traffic to bypass dl detector," in *Information and Communications Security: 23rd International Conference, ICICS 2021, Chongqing, China, November 19-21, 2021, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2021, p. 78–96. [Online]. Available: https://doi.org/10.1007/978-3-030-86890-1_5

[5] C. Fu, Q. Li, M. Shen, and K. Xu, "Detecting tunneled flooding traffic via deep semantic analysis of packet length patterns," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 3659–3673. [Online]. Available: https://doi.org/10.1145/3658644.3670353

[6] D. Brumley, J. Caballero, Z. Liang, J. Newsome, and D. Song, "Towards automatic discovery of deviations in binary implementations with applications to error detection and fingerprint generation." in *USENIX Security Symposium*, vol. 15, 2007.

[7] H. Dreger, A. Feldmann, M. Mai, V. Paxson, and R. Sommer, "Dynamic application-layer protocol analysis for network intrusion detection," in *15th USENIX security symposium*. USENIX Association, 2006, pp. 257–272.

[8] J. Chandler, A. Wick, and K. Fisher, "Binaryinferno: A semantic-driven approach to field inference for binary message formats." in *NDSS*, 2023.

[9] S. Kleber, H. Kopp, and F. Kargl, "{NEMESYS}: Network message syntax reverse engineering by analysis of the intrinsic structure of individual messages," in *12th USENIX Workshop on Offensive Technologies (WOOT 18)*, 2018.

[10] Z. Luo, K. Liang, Y. Zhao, F. Wu, J. Yu, H. Shi, and Y. Jiang, "Dynpre: Protocol reverse engineering via dynamic inference," in *Proc. NDSS*, 2024, pp. 1–18.

[11] Y. Ye, Z. Zhang, F. Wang, X. Zhang, and D. Xu, "Netplier: Probabilistic network protocol reverse engineering from message traces." in *NDSS*, 2021.

[12] J. Caballero, P. Poosankam, C. Kreibich, and D. Song, "Dispatcher: Enabling active botnet infiltration using automatic protocol reverse-engineering," in *Proceedings of the 16th ACM conference on Computer and communications security*, 2009, pp. 621–634.

[13] J. Caballero, H. Yin, Z. Liang, and D. Song, "Polyglot: Automatic extraction of protocol message format using dynamic binary analysis," in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 317–329.

[14] W. Cui, M. Peinado, K. Chen, H. J. Wang, and L. Irun-Briz, "Tupni: Automatic reverse engineering of input formats," in *Proceedings of the 15th ACM conference on Computer and communications security*, 2008, pp. 391–402.

[15] R. Droms, "Dynamic host configuration protocol," RFC 2131, March 1997. [Online]. Available: https://www.rfc-editor.org/info/rfc2131

[16] P. Brooks, "Ethernet/ip-industrial protocol," in *ETFA 2001. 8th International Conference on Emerging Technologies and Factory Automation. Proceedings (Cat. No. 01TH8597)*, vol. 2. IEEE, 2001, pp. 505–514.

[17] S. Viktor, D. Vangompel, and R. Voss, "The common industrial protocol (cip) and the family of cip networks," *Milwaukee, Wisconsin, USA, ODVA*, 2016.

[18] W. K. Wong, D. Wu, H. Wang, Z. Li, Z. Liu, S. Wang, Q. Tang, S. Nie, and S. Wu, "Decllm: Llm-augmented recompilable decompilation for enabling programmatic use of decompiled code," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1841–1864, 2025.

[19] P. Hu, R. Liang, and K. Chen, "Degpt: Optimizing decompiler output with llm," in *Proceedings 2024 Network and Distributed System Security Symposium*, vol. 267622140, 2024.

[20] X. She, Y. Zhao, and H. Wang, "Wadec: Decompiling webassembly using large language model," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 481–492.

[21] X.-L. Meng and D. Van Dyk, "The em algorithm—an old folk-song sung to a fast new tune," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 59, no. 3, pp. 511–567, 01 2002. [Online]. Available: https://doi.org/10.1111/1467-9868.00082

[22] "Rfc 1350: The tftp protocol (revision 2)," https://www.rfc-editor.org/rfc/rfc1350, 2025.

[23] "What properties, advantages and special features does the s7 protocol offer - siemens industry online support," https://support.industry.siemens.com/cs/document/26483647/what-properties-advantages-and-special-features-does-the-s7-protocol-offer-?dti=0&lc=en-WW, 2025.

[24] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 534–46 594, 2023.

[25] C. Hu, Y. Hu, H. Cao, T. Xiao, and J. Zhu, "Teaching language models to self-improve by learning from language feedback," *arXiv preprint arXiv:2406.07168*, 2024.

[26] J. Lu, W. Zhong, W. Huang, Y. Wang, F. Mi, B. Wang, W. Wang, L. Shang, and Q. Liu, "SELF: Language-driven self-evolution for large language model," 2024. [Online]. Available: https://openreview.net/forum?id=XD0PHQ5ry4

[27] X. Liang, S. Song, Z. Zheng, H. Wang, Q. Yu, X. Li, R.-H. Li, Y. Wang, Z. Wang, F. Xiong *et al.*, "Internal consistency and self-feedback in large language models: A survey," *arXiv preprint arXiv:2407.14507*, 2024.

[28] C.-H. Chiang and H.-y. Lee, "Can large language models be an alternative to human evaluations?" *arXiv preprint arXiv:2305.01937*, 2023.

[29] Z. Sun, Y. Shen, Q. Zhou, H. Zhang, Z. Chen, D. Cox, Y. Yang, and C. Gan, "Principle-driven self-alignment of language models from scratch with minimal human supervision," *Advances in Neural Information Processing Systems*, vol. 36, pp. 2511–2565, 2023.

[30] J. Fu, S.-K. Ng, Z. Jiang, and P. Liu, "Gptscore: Evaluate as you desire," *arXiv preprint arXiv:2302.04166*, 2023.

[31] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[32] E. J. Schwartz, T. Avgerinos, and D. Brumley, "All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask)," in *2010 IEEE symposium on Security and privacy*. IEEE, 2010, pp. 317–331.

[33] T. Moseley, D. Grunwald, D. A. Connors, R. Ramanujam, V. Tovinkere, and R. Peri, "Loopprof: Dynamic techniques for loop detection and profiling," in *Proceedings of the 2006 Workshop on Binary Instrumentation and Applications (WBIA)*, 2006.

[34] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *Advances in neural information processing systems*, vol. 36, pp. 11 809–11 822, 2023.

[35] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.

[36] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[37] Z. Lin, X. Jiang, D. Xu, and X. Zhang, "Automatic protocol format reverse engineering through context-aware monitored execution," 01 2008.

[38] "KTP," https://github.com/laksh0820/Custom-Transport-Protocol-Socket, 2025.

[39] "CCN," https://github.com/BenjaminWang236/Custom-Protocol-for-Client-Server-Communication, 2025.

[40] "Xfer," https://github.com/ZAK3IR/xfer, 2022.

[41] "Mirai," https://github.com/jgamblin/Mirai-Source-Code, 2025.

[42] S. Diao, P. Wang, Y. Lin, R. Pan, X. Liu, and T. Zhang, "Active prompting with chain-of-thought for large language models," *arXiv preprint arXiv:2302.12246*, 2023.

[43] M. Renze and E. Guven, "Self-reflection in llm agents: Effects on problem-solving performance, 2024," *URL https://arxiv.org/abs/2405.06682*, 2024.

[44] A. Slobodkin, O. Goldman, A. Caciularu, I. Dagan, and S. Ravfogel, "The curious case of hallucinatory (un) answerability: Finding truths in the hidden states of over-confident large language models," in

Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 3607–3625.
[45] M. Xiong, Z. Hu, X. Lu, Y. Li, J. Fu, J. He, and B. Hooi, "Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 23 650–23 678.
[46] W. H. Walters and E. I. Wilder, "Fabrication and errors in the bibliographic citations generated by chatgpt," *Scientific reports*, vol. 13, no. 1, p. 14045, 2023.
[47] N. Kandpal, H. Deng, A. Roberts, E. Wallace, and C. Raffel, "Large language models struggle to learn long-tail knowledge," in *International conference on machine learning*. PMLR, 2023, pp. 15 696–15 707.
[48] S. Lin, J. Hilton, and O. Evans, "Truthfulqa: Measuring how models mimic human falsehoods," in *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)*, 2022, pp. 3214–3252.
[49] Robert Bosch GmbH, *CAN Specification Version 2.0*, 1991, http://esd.cs.ucr.edu/webres/can20.pdf.
[50] "freemodbus," https://github.com/cwalter-at/freemodbus, 2025.
[51] "Open real-world traces for Modbus and DNS," https://www.netresec.com/?page=PcapFiles, 2025.
[52] "snap7," https://sourceforge.net/projects/snap7, 2025.
[53] "Open real-world traces for S7Comm," https://wiki.wireshark.org/SampleCaptures#s7comm-s7-communication, 2025.
[54] "OpENer," https://github.com/EIPStackGroup/OpENer, 2025.
[55] "Open real-world traces for Ethernet/IP," https://github.com/ITI/ICS-Security-Tools/blob/master/pcaps/EthernetIP/EthernetIP-CIP.pcap, 2025.
[56] "opendnp3," https://github.com/dnp3/opendnp3, 2025.
[57] "LightFTP," https://github.com/hfiref0x/LightFTP, 2025.
[58] "Open real-world traces for FTP," ftp://ita.ee.lbl.gov/new/lbnl.anon-ftp.03-01-10.tcpdump.gz, 2025.
[59] "tftp-hpa," https://git.kernel.org/pub/scm/network/tftp/tftp-hpa, 2025.
[60] "Open real-world traces for TFTP," https://www.cloudshark.org/captures/07ebe14c792b, 2025.
[61] "dnsmasq," https://github.com/infinet/dnsmasq, 2025.
[62] "miniweb," https://github.com/avih/miniweb, 2025.

APPENDIX

A. Prompt Design

Prompting. We use structured prompt templates to guide the LLM through field detection, sub-payload identification, and semantic inference. Beyond a fixed introduction, each prompt includes (i) taint and loop analysis outputs, (ii) the raw message, (iii) high-confidence semantic cues, and (iv) explicit reasoning instructions. This structure constrains the model to task-specific context, reducing hallucinations and improving consistency across runs. At initialization, we instruct the LLM to act as an expert in protocol reverse engineering.

1) The Prompt Template: Field Detection : The prompt template for the task of field detection comprises by five parts, shown in Figure 10, which are titled in italic font. We set Tree-of-Thoughts (ToT) parameters in the 'step-by-step instructions' section to guide LLM reasoning.

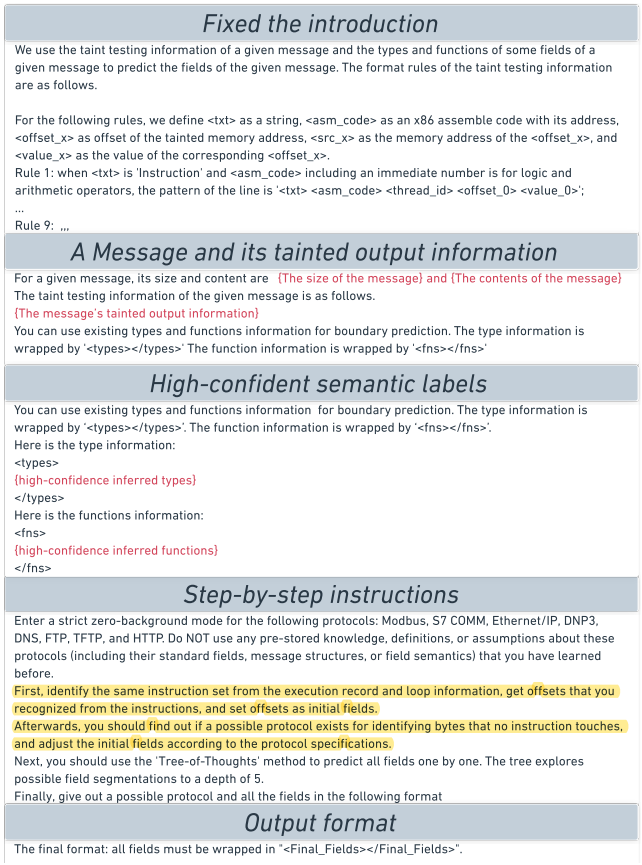


Fig. 10: Prompt Structure: Field Detection Template

2) The Conversation for Field Detection: We start a conversation with the LLM for field detection, shown in Figure 11. With the context, the LLM only reasons the untainted bytes.

3) The Prompt Template: Semantic Inference: The template for semantic inference comprises by four parts, as shown in Figure 12, which are titled in italic font. 'known information' includes a message, its fields, high-confident semantic labels.

4) The Conversation for Semantic Inference: In each iteration, we apply the field-detection and semantic-inference prompts to obtain updated fields and semantic labels. The

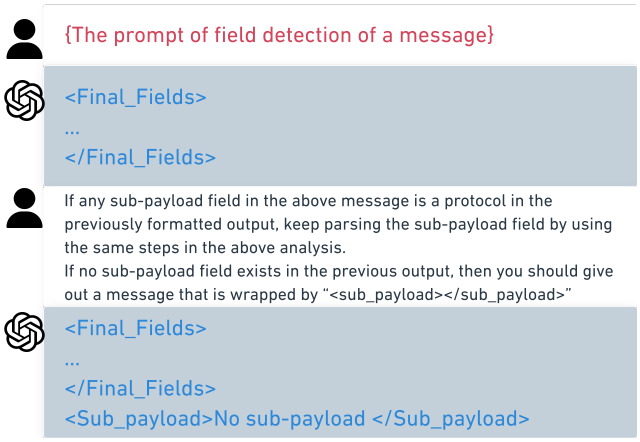


Fig. 11: LLM Interaction: Field Detection Process

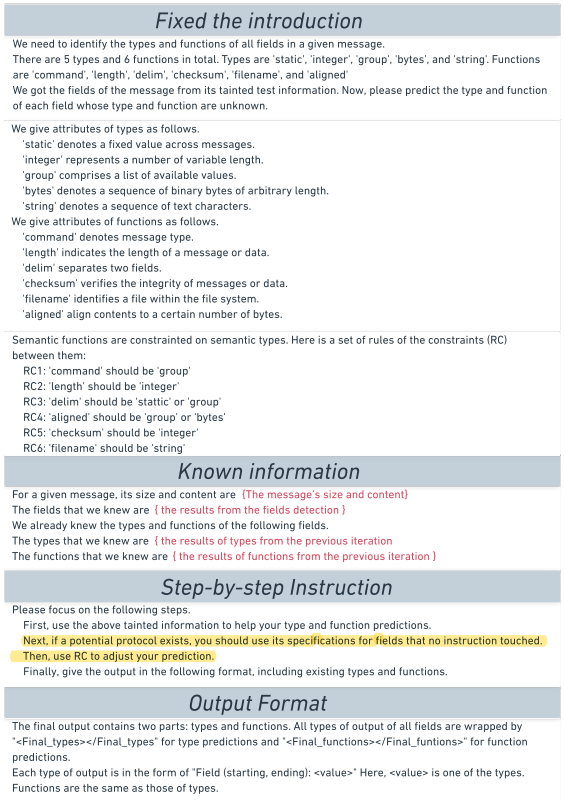


Fig. 12: Prompt Structure: Semantic Inference Template

taint information continues to guide the LLM by enforcing consistency between the detected fields and their inferred semantics. The structure is in Figure 13

B. Tables

C. Common Types and Functions

We follow the definitions of types and functions as identified in prior work [2]. Details of definitions are in Table XI

1) Results from Different Tools: We evaluate the examples from the section II-B using multiple tools and summarize the results in Tables XII–XIII–XIV. We compare SEMPRE against

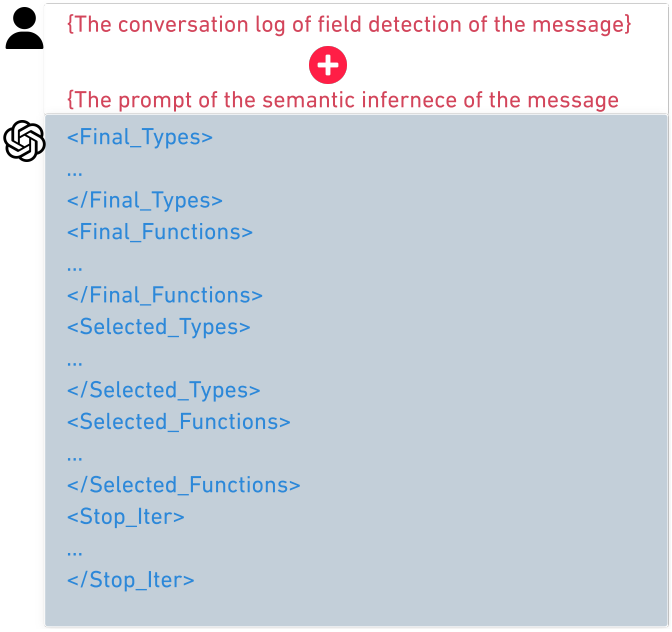


Fig. 13: LLM Interaction: Semantic Inference Process

TABLE XI: Semantic types and functions

Semantic Types	
Static	Fixed values across messages, usually protocol-specific flags
Integer	Numeric metadata (e.g., sizes), in either big- or little-endian
Group	Enumerated static values for actions, message types, etc.
Bytes	Protocol data in raw binary format
String	Text character sequences (e.g., file names)
Semantic Functions	
Command	Denotes message types
Length	Indicates the length of messages or data
Delim	Separates two fields
Checksum	Verifies message or data integrity
Filename	Identifies a file within the file system
Aligned	Aligns contents to a specified byte boundary

BinPRE, Polyglot, and Tupni (BPT) to highlight both the strengths of SEMPRE and the limitations of BPT. As shown in Table XII, the results illustrate the shortcomings of BPT's inflexible, heuristic-based rules.

Next, we observe the effects of the ignorance of semantic relationships in BPT. However, SEMPRE can correctly rebuild them in Table XIII.

Finally, we spot the power of SEMPRE about detecting sub-payload but BPT can't in Table XIV

2) Details for Protocol in Datasets: Table XV shows the details of the protocols in the datasets. Format colomn indicates the categories of the content of procotol messages: binary, text, or mixed. Project colomn shows the resources of the protocol binary file.

TABLE XII: Different Results on the Filename Field of TFTP from Different Tools

Tools	Field	Function
BinPRE	[2], [3, 17]	None, Aligned
Polyglot	[2], [3], [4], ..., [16], [17]	-
Tupni	[2], [3, 17]	-
SEMPRE	[2, 17]	Filename
GroundTruth	[2, 17]	Filename

TABLE XIII: Different Results on Two Length Fields of S7 from Different Tools

Tools	Field	Function
BinPRE	△	None, None
Polyglot	△	None, None
Tupni	△	None, None
SEMPRE	[2, 3], [4]	Length, Length
GroundTruth	[2, 3], [4]	Length, Length

△: incorrect field detection; None: the lack of prediction

TABLE XIV: Different Results on Nested Protocol of EIP from Different Tools

Tools	Field	Nested Protocol
BinPRE	[30, 81]	-
Polyglot	△	-
Tupni	△	-
SEMPRE	[46, 81]	D.
GroundTruth	[46, 81]	CIP

△: incorrect field detection: -: not support; D.: detect a nested sub-payload;

TABLE XV: Protocol Datasets for Evaluation.

Dataset	Protocol	Format	Project	Message Source
D1	Modbus	binary	`freemodbus` [50]	Open-source [51]
	S7comm	binary	`snap7` [52]	Open-source [53]
	Ethernet/IP	binary	`OpENer` [54]	Open-source [55]
	DNP3.0	binary	`opendnp3` [56]	`tcpdump`
	FTP	text	`LightFTP` [57]	Open-source [58]
	TFTP	mixed	`tftpd-hpa` [59]	Open-source [60]
	DNS	mixed	`dnsmasq` [61]	Open-source [51]
	HTTP	text	`miniweb` [62]	`tcpdump`
D2	KTP	binary	`KTP-Source-Code` [38]	`tcpdump`
	CCN	binary	`CCN-Source-Code` [39]	`tcpdump`
	Xfer	mixed	`Xfer-Source-Code` [40]	`tcpdump`
D3	Mirai	mixed	`Mirai-Source-Code` [41]	`tcpdump`

3) Semantic Capabilities of the Baselines: We summarize the semantic modeling capabilities of representative prior protocol reverse engineering (PRE) systems. We compare these approaches along three dimensions: (1) the analysis paradigm they adopt, (2) the kinds of semantic types for protocol fields they infer, and (3) the kinds of semantic functions they are

able to identify. Details are in Table XVI.

TABLE XVI: Capability of baselines and SEMPRE

Protocols	Approaches	Types	Functions
PolyGlot	binary-analysis-based	-	CMD; LEN; DELIM
Tupni	binary-analysis-based	-	LEN; CHS
AutoFormat	binary-analysis-based	-	-
BinPRE	binary-analysis-based	$COMM_{Types}$	$COMM_{Fns}$
SEMPRE	binary-analysis-based	$COMM_{Types}$	$COMM_{Fns}$
BinaryInferno	network-traffic-based	-	LEN
DynPRE	network-traffic-based	-	CMD, LEN

-: not support; CMD: command; LEN: length; DELIM: delimitor; CHS: checksum; $COMM_Types$: common types; $COMM_Fns$: common functions.

4) Tables Related to RQ3: Table XVII shows the results for SEMPRE with different backbones. Results show that the performance of SEMPRE does not depend on a specific LLM model.

TABLE XVII: Performance with Different Backbones.

Protocols	SEMPRE-o3			SEMPRE-ds			SEMPRE-hy		
	Field	Type	Fn.	Fields	Type	Fn.	Fields	Type	Fn.
Modbus	**0.99**	**0.81**	**1.00**	0.98	0.74	0.98	0.93	0.61	0.81
S7comm	**0.81**	**0.80**	**0.64**	0.64	0.39	0.26	0.76	0.49	0.31
Ethernet/IP	**0.80**	**0.69**	**1.00**	0.42	0.64	0.97	0.46	0.65	**1.00**
DNP3	**0.80**	**0.82**	**0.90**	0.65	0.47	0.42	0.61	0.37	0.25
DNS	0.72	**0.71**	**0.93**	**0.75**	0.64	0.73	0.71	0.46	0.58
FTP	**0.66**	**0.72**	**0.77**	0.63	0.53	0.66	0.64	0.32	0.59
TFTP	**1.00**	0.35	**1.00**	0.71	0.53	0.95	0.68	**0.56**	0.81
HTTP	**0.60**	**0.62**	**0.70**	**0.60**	**0.62**	**0.70**	0.11	0.01	0.03
Average	**0.76**	**0.72**	**0.74**	0.58	0.59	0.62	0.49	0.39	0.46

SEMPRE-o3: the backbone of SEMPRE is GPT-o3;

SEMPRE-ds: the backbone of SEMPRE is DeepSeek-v3.1-terminus

SEMPRE-hy: the backbone of SEMPRE is Hy3

Table XVIII shows the results on undocumented protocols under different approaches.

TABLE XVIII: Effectiveness on Undocumented Protocols.

Protocols	SEMPRE			BinPRE			Vanilla GPT-o3		
	Field	Type	Fn.	Fields	Type	Fn.	Fields	Type	Fn.
KTP	0.50	0.22	0.44	**0.44**	0.00	0.00	**0.57**	**0.24**	0.13
CCN	**1.00**	**0.72**	**0.92**	0.53	0.50	0.00	0.48	0.38	0.67
Xfer	**0.73**	**0.49**	**0.93**	0.22	0.18	0.29	0.47	0.38	0.40
Avg.	**0.71**	**0.43**	**0.77**	0.41	0.19	0.11	0.52	0.31	0.43

5) Tables Related to RQ2: Table XIX show the details of metrics from SEMPRE and baselines. DynPRE receives and responds to a subset of messages for S7comm and DNS, so the results depend on the messages it responds to.

TABLE XIX: Function Inference Results.

Protocols	SemPRE			BinPRE			Polyglot			Tupni			Binaryinferno			DynPRE		
	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1
Modbus	1.00	1.00	**1.00**	1.00	1.00	**1.00**	0.64	1.00	0.78	1.00	0.50	0.67	0.00	0.00	0.00	0.00	0.00	0.00
S7comm	0.78	0.55	**0.64**	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00[1]	0.00[1]	0.00[1]
Ethernet/IP	1.00	0.50	**0.67**	1.00	0.10	0.17	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	N/A	N/A	N/A
DNP3	0.99	0.83	**0.90**	1.00	0.75	0.86	0.25	0.25	0.25	1.00	0.25	0.40	0.00	0.00	0.00	N/A	N/A	N/A
DNS	1.00	0.86	**0.93**	0.80	0.71	0.75	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00[2]	0.00[2]	0.00[2]
FTP	1.00	0.63	**0.77**	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	N/A	N/A	N/A
TFTP	1.00	1.00	**1.00**	1.00	0.47	0.64	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	N/A	N/A	N/A
HTTP	0.99	0.54	**0.70**	0.00	0.00	0.00	0.00[3]	0.24	0.00[4]	0.00	0.00	0.00	0.00	0.00	0.00	N/A	N/A	N/A
Average	0.96	0.61	**0.74**	0.82	0.22	0.35	0.01	0.09	0.01	0.00[5]	0.04	0.00[6]	0.00	0.00	0.00	0.00	0.00	0.00

Average - The performance on all protocols; Bold numbers indicate the best results
1. counting based on the first 31 messages; 2 - counting based on the first 16 messages; 3 - 0.0015; 4 - 0.0030; 5 - 0.0026; 6 - 0.0050